arXiv:submit/8337151 [cs.RO] 6 Nov 2026

LODOS: A Lightweight Simulator Designed for Contact-Rich Interactions with Detailed Meshes

Onur Beker[1], A. René Geist[1], Georg Martius[1,2]
[1]University of Tübingen, [2]Max Planck Institute for Intelligent Systems

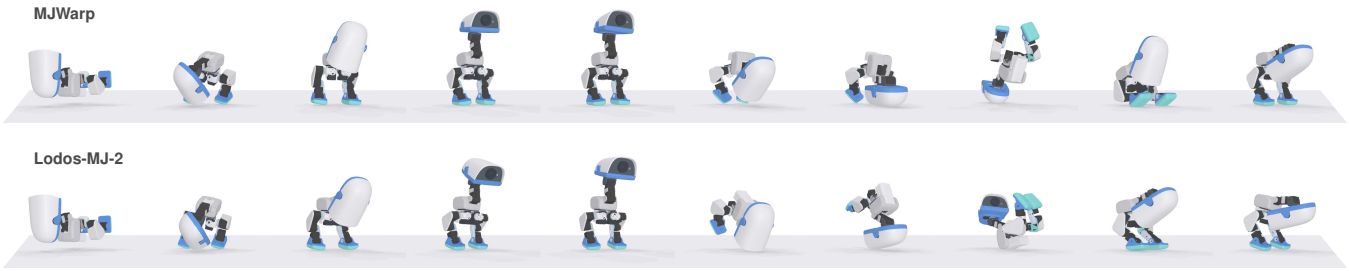


Fig. 1: A Microduck getting up and rolling, simulated in MJWarp and Lodos. The policy is trained in MJWarp and transferred zero-shot.

***Abstract*— In recent years, massively parallelized rigid-body simulation has played a central role in robotics research, enabling rapid progress in areas such as reinforcement learning, stochastic optimal control, and randomized smoothing-based trajectory optimization. Particularly in contact-rich settings, the scalability of such simulations in large part depends on well-optimized representations of collision geometry, often requiring a coarse approximation of the underlying surfaces using primitives such as spheres, capsules, cylinders, and boxes. Whenever the complexity of the surfaces or the precision demands of the application render such approximations insufficient, it is instead necessary to employ mesh-mesh collisions, which impose a significant reduction in throughput. This paper proposes a simulation framework designed from scratch to address this bottleneck, with novel routines for broadphase and narrowphase collision detection as well as the subsequent contact dynamics computation. As experimentally validated, the framework matches existing well-established simulators in terms of physical faithfulness, and can achieve approximately an order of magnitude higher throughput in the detailed mesh-mesh collision setting that it specializes in. The source code is available at https://github.com/martius-lab/lodos.**

I. Introduction

What makes a good simulator for solving control problems that involve contact-rich interactions? Let us build the desiderata by focusing on the control and contact-rich interaction parts of the question separately.

A good simulator for control: Sampling-based control approaches that leverage massively vectorized simulation [1, 2] have proven to be very proficient at solving contact-rich manipulation tasks. A common pattern shared by such approaches, whether it is domain randomization with RL [3, 4], risk-aware control with stochastic MPC [5], or randomized smoothing [6] with gradient-based optimization, is that actions are selected such that they remain "approximately correct" with respect to metrics averaged over randomized ensembles of contact parameters generated via artificial (and oftentimes non-physical) perturbations (e.g. adding virtual forces, noisy friction coefficients and haptic sensing, approximate or domain randomized collision geometry). And a main argument of this paper is that in such a setting, perturbations and averaging already introduce a "convex-hull" effect on how much the physical faithfulness of each ensemble member can reflect on (or matter for) the selection of the common actions facing the ensemble. Therefore, there should be room for using simplified versions of contact dynamics (that can still sufficiently preserve the "convex-hull") to achieve drastically increased ensemble sizes instead, resulting in a net benefit for the downstream control algorithm.

A good simulator for contact-rich interactions: Most of the commonly used simulators in robotics research [7, 8, 9, 10] use a combination of convex polytope based collision detection [11, 12] with a numerical optimization routine to solve the regularized non-linear complementarity (NCP) or cone-complementarity (CCP) systems of equations describing contact dynamics [13, 14, 15, 9, 16, 17]. While this battle-tested recipe certainly has its merits, it suffers from the following drawbacks particularly in contact-rich settings:

- The regularization introduced to the NCP/CCP equations scales frictional contact forces down from the exact value needed to maintain complete staticity in a rigid-body system (e.g. zero velocity and acceleration), meaning all contacts eventually slip under constant forces such as gravity.
- Applying GJK-EPA algorithms [11, 18] to convex polytope collisions produces a single contact point, which is occasionally further extended into a sparse set of contact points lying at the corners of a 2D polygon [12]. Neither mechanism can faithfully implement contact forces distributed over a patch, meaning torsional friction is emulated in an ad hoc manner by introducing artificial constraints to the optimization routine that oppose motion around the contact normal.
- Convex-polytope-based collision routines scale poorly with parallelization, particularly under contact-rich configurations involving complex non-convex meshes.

In concert, these drawbacks chart a path toward the design of an effective simulator for contact-rich interactions.

Contributions. We present a novel simulation routine designed from scratch to facilitate research on developing control algorithms in contact-rich settings. In particular:

- We propose a new collision detection routine built upon

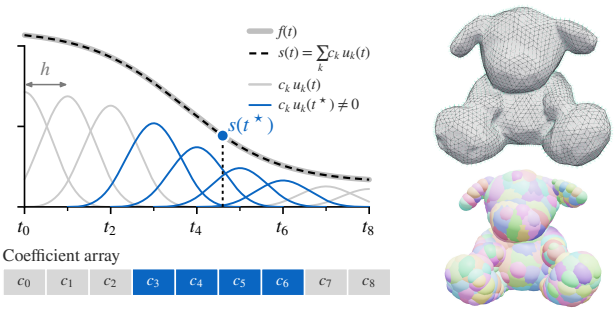


Fig. 2: Left: Cubic spline interpolation. In 1D, evaluating the spline $s(t^\star)$ requires accessing only four entries in the coefficient array (more generally 4^N in N-D). **Right:** A collision geometry is represented via two pieces of information: i) an SDF implemented with a 3D spline, and ii) zero volume spheres (i.e. vertices) densely distributed on the 2D surface of the mesh and inscribed spheres distributed within the 3D volume of the mesh.

sphere - SDF collisions as the axiomatic interaction, which can represent arbitrary non-convex (closed) surfaces at arbitrary levels of precision, and scales much more favorably with massive parallelization compared to the traditional convex polytope based approach. Specifically, this includes a novel hierarchical broadphase routine to logarithmically cull potential sphere-SDF interactions, and a novel narrowphase routine using 3D-spline-based SDF and surface normal representations whose evaluation time is independent of geometry complexity.

- We propose a new model of contact dynamics based on rescaling spring-damper forces via a staggered fixed-point iteration scheme, derived from a lumped parameterization of the traditional SOCP-based contact models employed in most common simulators [16, 7]. The main benefits are that it: i) scales much more favorably under massive vectorization compared to standard approaches, ii) does not exhibit a slipping artifact, iii) allows patch-patch contacts and therefore faithful torsional friction simulation.

II. Related Work

Rigid-body simulation with contact is central to robotics. Classical methods, still used in CAD software [19], model contact with dissipative spring-dampers [13, 20, 21], yielding stiff ODEs that require slow, time-adaptive integration. Discrete time-stepping methods [22, 17] instead jointly determine contact forces and post-impact velocities through the NCP equations [15]. Appropriate relaxations yield efficient convex approximations [16], as used in MuJoCo [7]. Alternatively, fixed-point iterations with a "Signorini correction" can solve the original NCP system [9], as in Simple [23]. A recent, complementary direction is the implementation of such models of contact dynamics on the GPU, to create massively parallelized simulators such as IsaacGym [1], MJWarp [2], and Kamino [10]. This paper shares the same goal as such frameworks: to maximize simulator throughput. However, unlike these frameworks that aim to provide high-quality GPU implementations while remaining completely faithful to existing, well-established models of contact dynamics (that were not necessarily formulated with massive parallelization in mind), the proposed approach sets out to create a new contact model that was designed from scratch with massive parallelization as the main concern, specifically in settings involving collisions between detailed non-convex meshes.

III. Methods: Collision Detection

Sphere-SDF collisions are the axiomatic interaction around which the entire collision detection stack is built. Therefore, we start with a description of how we represent SDFs.

A. Efficiently Representing SDFs with 3D Splines

1) Cubic Splines in 3D: In 1D, standard cubic splines are built as a sum of twice-differentiable polynomial basis functions $u : \mathbb{R} \to \mathbb{R}$ of the following form [24]:

$$u(t) = \begin{cases} 4 - 6|t|^2 + 3|t|^3 & \text{for } |t| \leq 1 \\ (2 - |t|)^3 & \text{for } 1 < |t| \leq 2 \\ 0 & \text{for } 2 < |t| \end{cases} \tag{1}$$

Given the values $\{f_i\}_{i=0}^{N}$ of a function $f : [a,b] \to \mathbb{R}$ evaluated at the corners of N intervals splitting $[a,b]$ into equal pieces of length h each, the cubic spline interpolation in 1D is given by the sum $s(t) = \sum_{k=1}^{N+3} c_k u_k(t)$, using a superposition of $N+3$ shifted basis functions defined as $u_i(t) = u(\frac{t-a}{h} - k + 2)$. The coefficients c_k are in turn obtained by solving the linear system defined by the equations $s(t_i) = f_i$. Given the values $\{f_{i,j,k}\}_{i,j,k=0}^{N_x,N_y,N_z}$ of a function $f : \mathbb{R}^3 \to \mathbb{R}$ evaluated at the corners of a 3D voxel grid with (N_x, N_y, N_z) voxels along each axis, a 3D spline interpolation is obtained as $s(x,y,z) = \sum_{k_x=0}^{N_x+3} \sum_{k_y=0}^{N_y+3} \sum_{k_z=0}^{N_z+3} c_{k_x,k_y,k_z}\, u_{k_x}(x) u_{k_y}(y) u_{k_z}(z)$, using products of the form $u_{k_x}(x)u_{k_y}(y)u_{k_z}(z)$ to define 3D basis functions from 1D basis functions. The coefficients c_{k_x,k_y,k_z} are again obtained by solving the linear system defined by the equations $s(x_i, y_j, z_k) = f_{i,j,k}$. The compact support of $u(t)$ ($u(t) = 0$ outside $[-2,2]$) makes evaluation fast and independent of grid size. In 3D, only 64 terms of $s(x,y,z)$ are nonzero at any query point, and their indices can be identified via integer rounding with minimal computational cost.

2) Using 3D Splines to Represent SDFs and Normals: We interpolate mesh signed distances sampled at voxel corners with a 3D cubic spline. Its normalized gradient (computed in closed form) defines the surface normal function (SNF). Compact basis support makes SDF and SNF evaluation costs independent of geometry complexity, which affects only the required grid resolution and storage. The grid is shared across parallel instances through concurrent GPU reads, so its storage does not grow with batch size.

B. Narrowphase Collision Detection

The goal of the narrowphase is to generate a set of contact points together with penetration depths and contact normals. Let us start with a description of the axiomatic sphere-SDF collision, around which the rest of the routine is built.

1) Sphere-SDF Collisions: Let $\boldsymbol{c} \in \mathbb{R}^3$ be the center of a sphere with radius $r \in \mathbb{R}$, colliding with a surface with the SDF $\phi(\boldsymbol{x}) : \mathbb{R}^3 \to \mathbb{R}$ and SNF $\boldsymbol{n}(\boldsymbol{x}) : \mathbb{R}^3 \to \mathbb{R}^3$. The contact point (lying on the surface of the sphere) is $\boldsymbol{p} = \boldsymbol{c} - \boldsymbol{n}(\boldsymbol{c})r$, with the contact depth $d = \phi(\boldsymbol{c}) - r$ (negative for penetration) and contact normal $\boldsymbol{n} = \boldsymbol{n}(\boldsymbol{c})$. An important observation is that a vertex–SDF collision [25] is strictly a special case of a sphere-SDF collision with $r = 0$.

2) Narrowphase: As shown in Fig.2, in the proposed framework, every collision geometry is represented by three pieces of information: a set of K spheres $\mathscr{S} = \{(\boldsymbol{c}_k, r_k)\}_{k=1}^{K}$ that are distributed across its surface and inscribed within its volume, together with an SDF $\phi(\boldsymbol{x})$ and an SNF $\boldsymbol{n}(\boldsymbol{x})$. Collision pairs are asymmetric: when a geometry i is colliding with a geometry j (forming a directional collision pair (i, j)), it means the spheres $\mathscr{S}_i$ are colliding with the SDF and SNF $\phi_j, \boldsymbol{n}_j$ to produce $|\mathscr{S}_i|$ contact points $\boldsymbol{p}_{i,j,k}$ with depths $d_{i,j,k}$ and normals $\boldsymbol{n}_{i,j,k}$. Symmetry, if desired, is achieved by defining two collision pairs (i, j) and (j, i) in both directions.

Naturally, the fidelity of the scheme depends on how well the spheres are distributed. The key insight is the following. Exact mesh-SDF collisions can be implemented by considering all vertex–SDF and edge–SDF contacts [25], and vertex–SDF contacts are already covered as a special case of sphere-SDF contacts. Therefore, we split the sphere placement $\mathscr{S}$ into two groups: i) zero-volume spheres (i.e. vertices, $r = 0$) densely distributed on the 2D surface of the mesh (i.e. implementing vertex–SDF contacts), and ii) inscribed spheres distributed within the 3D volume of the mesh to cover as much volume as possible. To create the former group, we employ the pipeline from [25] to obtain a dense isotropic remesh and take the resulting vertices. The latter group can be constructed manually [26] (the option we opt for), or any sphere decomposition algorithm can be used [27]. It is also important to note that there is no distinction between how the two different groups of spheres are processed by the software implementation of the broadphase and narrowphase routines (the former group simply has all radii set to $r = 0$).

3) Why omit edge–SDF contacts? In the proposed framework, vertex–SDF and inscribed-sphere–SDF collisions are already exact, but omitting edge–SDF contacts can in theory admit millimeter-scale penetrations into the gaps between surface vertices and underlying spheres. This artifact can be detected and continuously measured by evaluating inscribed-sphere contact points against their own geometry's SDF (i.e. contacts should not lie far beneath that surface). It arises only: i) with pathological vertex distributions (e.g., coarse non-isotropic meshes), and ii) in edge–edge interactions, such as two cube edges meeting. Such edge-edge interaction modalities are inherently unstable and therefore undesirable for manipulation. And a controller can avoid them by penalizing the penetration measure. From a *physics-for-control perspective*, this is preferable to a $\sim$2–4$\times$ slowdown for edge–SDF detection that suppresses a minor, detectable, and avoidable artifact associated with undesirable behavior.

C. Broadphase Collision Detection

Without any broadphase, narrowphase involves the spheres $\mathscr{S}_i$ of a geometry i colliding with the SDF and SNF $\phi_j, \boldsymbol{n}_j$ of another geometry j to produce $|\mathscr{S}_i|$ contact points. And while the speed and computational cost of $\phi_j, \boldsymbol{n}_j$ do not depend on the geometry complexity, the number of spheres $|\mathscr{S}_i|$ does and it can be as large as $\sim 10^4$ for very complex surfaces. The key observation that allows broadphase is that by definition, contact interactions keep the intersection between colliding surfaces minimal, meaning the overwhelming majority of the $|\mathscr{S}_i|$ contact points have positive signed distances and hence do not affect the underlying physics. These non-penetrating contact points can easily be detected and culled in an $\mathscr{O}(\log|\mathscr{S}_i|)$ manner using a hierarchical culling scheme as described in the upcoming sections.

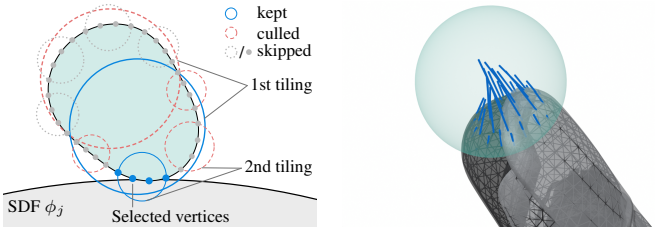


Fig. 3: Left: Selecting vertices for the narrowphase using tiled sphere hierarchies. **Right:** The resulting dense contact force field.

1) Voxelized SDF: To enable the broadphase, in addition to the voxel-corner samples used for spline interpolation, we store signed distances at voxel centers for $\mathscr{O}(1)$ lookup. For a point $\boldsymbol{p}$ at distance Δd from its voxel center $\boldsymbol{p}_c$, $\phi(\boldsymbol{p}) + \Delta d \geq \phi(\boldsymbol{p}_c)$ holds [28]. If r_v is the voxel circumsphere radius, then $\phi(\boldsymbol{p}_c) - r_v$ bounds $\phi(\boldsymbol{p})$ from below. Thus, a sphere of radius r centered at $\boldsymbol{p}$ can be culled whenever $\phi(\boldsymbol{p}_c) - r_v \geq r$.

2) Tiled Sphere Hierarchies: To enable efficient culling of spheres $\mathscr{S}_i$ using the voxelized SDF checks from the previous section, we need to build a hierarchical acceleration structure. This is achieved as follows. First, we serialize sphere centers along a 3D Hilbert space-filling curve, a standard locality-preserving technique in geometry processing [29, 30]. This ensures that if the centers of two spheres are close in terms of 3D Euclidean distance, then their indices are also close together after reordering. Then, T consecutive spheres are grouped into a single "tile" (resulting in $\sim |\mathscr{S}_i|/T$ tiles), and the smallest bounding sphere of every tile is computed and stored. The number T is fixed for every tile to make reindexing and stream compaction more efficient. In principle, this process of serialization and tiling can be repeated an arbitrary number of times to construct a tile hierarchy. In practice, we found two levels to be sufficient for geometries with several thousand spheres. The broadphase traverses the hierarchy from the highest level. At each level, voxelized SDF checks cull tiles, and stream compaction records surviving indices for the next level. Since the dimensions of stream compacted arrays depend on geometry poses, all variable-size scans use a "reduce-then-scan" strategy [31]. Surviving leaf spheres then enter the narrowphase.

3) Preventing Host-Device Synchronization: While the reduce-and-scan strategy allows variable-length scans to run entirely within the GPU without any CPU communication, the memory that stores the arrays being scanned still needs to be preallocated at compile time to prevent host-synchronization and allow CUDA graph capture [32]. And since an overwhelmingly small percentage of all sphere-SDF contacts is active during a simulation, allocating memory for the worst-case scenario of $|\mathscr{S}_i|$ contacts per simulation instance would be drastically wasteful. Therefore, the "capacities" that determine how much memory needs to be allocated (as a percentage of the estimates for all $|\mathscr{S}_i|$ contacts) are calibrated live as the simulation is running, via a simple control rule that monitors the largest number of contacts C observed so far in the lifetime of a simulation. At the beginning of the simulation, $C = |\mathscr{S}_i|$

and all capacities start at %100. At regular intervals, they are readjusted to support αC contacts (with the headroom set to $\alpha = 2$ by default) if there is sufficient change in C. Anytime an overflow is detected, this again triggers the same capacity readjustment. Naturally, every such re-adjustment triggers a host-device synchronization, but this auto-calibration tends to quickly converge to a stable state.

4) Oriented Bounding-Box Prepass: Before culling spheres within each pair (i, j), an oriented bounding-box (OBB) prepass rejects entire collision pairs.

IV. Methods: Contact Dynamics

With the collision detection routine covered, this section focuses on the proposed contact dynamics model. We start with a compressed summary of how commonly used simulators in robotics such as MuJoCo [7] and Drake [8] operate.

A. Background

The dynamics of a rigid-body system with generalized position $\mathbf{q} \in \mathbb{R}^{n_q}$, generalized velocity $\mathbf{v} \in \mathbb{R}^{n_v}$, and generalized acceleration $\mathbf{a} \in \mathbb{R}^{n_v}$, under a set of C separate external forces $\mathbf{f} \in \mathbb{R}^{3C}$ (concatenated into a single vector) is governed by the Newton-Euler equation:

$$\mathbf{M}(\mathbf{q})\mathbf{a} + \mathbf{b}(\mathbf{q}, \mathbf{v}) = \boldsymbol{\tau}_u + \mathbf{J}(\mathbf{q})^\top \mathbf{f} \tag{2}$$

where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n_v \times n_v}$ is the generalized inertia matrix, $\mathbf{b}(\mathbf{q}, \mathbf{v})$ is the generalized bias force, $\boldsymbol{\tau}_u$ is the generalized force due to actuators, and $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{n_v \times 3C}$ is the concatenation of the Jacobians $\mathbf{J}_i \in \mathbb{R}^{3 \times n_v}$ computed at the application points of each individual force $\boldsymbol{f}_i \in \mathbb{R}^3$ within the concatenated force vector $\mathbf{f} \in \mathbb{R}^{3C}$. The expressions $\mathbf{a}_f = \mathbf{M}^{-1}(\boldsymbol{\tau}_u - \mathbf{b})$ and $\mathbf{v}_f = \mathbf{v} + \mathbf{a}_f \Delta t$ denote the free acceleration (i.e. the generalized acceleration in the absence of any external forces) and free velocity (i.e. the generalized velocity after Euler-integration with timestep Δt and acceleration $\mathbf{a}_f$). The generalized acceleration in the presence of external forces can in turn be computed as $\mathbf{a} = \mathbf{M}^{-1}(\boldsymbol{\tau}_u - \mathbf{b}) + \mathbf{M}^{-1}\bar{\mathbf{J}}^\top \mathbf{f} = \mathbf{a}_f + \mathbf{M}^{-1}\bar{\mathbf{J}}^\top \mathbf{f}$. The post-impact generalized velocity $\mathbf{v}^+$ is then defined via Euler integration as $\mathbf{v}^+ = \mathbf{v} + \mathbf{a}\Delta t = \mathbf{v}_f + (\mathbf{M}^{-1}\bar{\mathbf{J}}^\top)\mathbf{f}\Delta t$. This means concatenated post-impact contact velocities $\bar{\boldsymbol{v}}^+ \in \mathbb{R}^{3N}$ are in turn defined as $\bar{\boldsymbol{v}}^+ = \bar{\mathbf{J}}\mathbf{v}^+ = \bar{\boldsymbol{v}}_f + (\bar{\mathbf{J}}\mathbf{M}^{-1}\bar{\mathbf{J}}^\top)\mathbf{f}\Delta t = \bar{\boldsymbol{v}}_f + \mathbf{G}\mathbf{f}\Delta t$, where $\mathbf{G} = \bar{\mathbf{J}}\mathbf{M}^{-1}\bar{\mathbf{J}}^\top \in \mathbb{R}^{3N \times 3N}$ is the Delassus matrix.

MuJoCo [7] and Drake [14] solve a second-order cone program (SOCP) to obtain contact forces:

$$\min_{\mathbf{f}} \frac{1}{2}\mathbf{f}^\top [(\mathbf{G} + \mathbf{R})\Delta t]\mathbf{f} + (\bar{\boldsymbol{v}}_f + \boldsymbol{\gamma})^\top \mathbf{f} \quad \text{s.t.} \quad \mathbf{f} \in \mathbb{K}_\mu \tag{3}$$

where the regularization matrix $\mathbf{R}(\mathbf{q}, \mathbf{v}) \in \mathbb{R}^{3C \times 3C}$ ensures that the objective is strictly positive definite, and the term $\boldsymbol{\gamma}(\mathbf{q}, \mathbf{v}) \in \mathbb{R}^{3C}$ implements penetration resolution. When $\mathbf{R} = \mathbf{0}$ and $\boldsymbol{\gamma} = \mathbf{0}$, the optimization problem reduces to finding contact forces within the friction cone that minimize post-impact kinetic energy of the rigid-body system after a semi-implicit Euler update (without resolving any existing penetrations, as this would require injecting energy into the rigid-body system). The proposed contact dynamics formulation described in the upcoming sections can work with arbitrary ways of setting $\mathbf{R}$ and γ, and the software implementation provides the MuJoCo [7] and kinetic energy minimization (i.e. $\mathbf{R} = \mathbf{0}$, $\boldsymbol{\gamma} = \mathbf{0}$) conventions by default.

B. Spring-Damper Based Contact Models

Unlike the optimization-based time-stepping methods of robotics and graphics, simulators commonly used in mechanical engineering [19] express contact forces in closed form as smooth functions of state and integrate the ODE adaptively [21]. These closed form expressions often take a functional form analogous to a spring-damper with dissipation (hence our naming convention for this contact model class), and are well-validated through empirical measurements in the real world [21, 33].

The main downside of the spring-damper based approach is that time-adaptive integration does not readily lend itself to massive vectorization, and rollouts have to progress and terminate at the speed of the stiffest simulation instance in the batch [34]. And without time-adaptive integration, creating a stable simulation requires careful hand-tuning for every new scene and interaction modality (e.g. anticipating the motions that will be involved, states that may be visited, and the resulting inertias that will be reflected on each contact point). On the other hand, with discrete time-stepping methods (to quote Todorov et al. [35] verbatim) "one can think of the contact impulses as being generated by smart spring-dampers, which are aware of each other and furthermore scale their stiffness and damping automatically with inertia". As explained in the next section, this intuition forms the basis of the contact models proposed in this paper, which operate by computing spring-damper based contact forces between collision pairs and then rescaling them (with a decision variable per collision pair) in a way that optimizes the original SOCP objective (3). In particular, the proposed framework provides two spring-damper based contact model options for this purpose: i) the smoothed linear spring model with Hunt and Crossley dissipation as proposed in [36], and ii) the "complementarity-free" spring-damper model of [37] with inertia-aware stiffness and damping terms.

C. Proposed Contact Model

Let us continue our exposition with the mathematical details of the proposed contact model.

1) Rescaling Spring-Damper Forces: Let us denote the set of all collision pairs as $\mathscr{K}$, indexed with $k \in \{1, \cdots, |\mathscr{K}|\}$. Let us further denote the total number of contact points in each collision pair as C_k. Finally, let us use the notation $\boldsymbol{f}_{k,l}(\mathbf{q}, \mathbf{v}) : \mathbb{R}^{n_q + n_v} \to \mathbb{R}^3$ to denote the spring-damper contact force computed for the l'th contact point of the collision pair k, as a function of system position and velocity. As previously defined, $\mathbf{f} \in \mathbb{R}^{3C}$ denotes the concatenation of individual contact forces $\boldsymbol{f}_{k,l} \in \mathbb{R}^3$, with the total number of contacts satisfying $C = \sum_{k=1}^{|\mathscr{K}|} C_k$. It is then possible to transcribe a lumped parameterization of $\mathbf{f}$ in terms of per-collision-pair scaling factors $\boldsymbol{\alpha} \in \mathbb{R}^{|\mathscr{K}|}$ as $\mathbf{f}(\boldsymbol{\alpha}) = \mathbf{F}\boldsymbol{\alpha}$, where $\mathbf{F}(\mathbf{q}, \mathbf{v}) \in \mathbb{R}^{3C \times |\mathscr{K}|}$ simply consists of the vectors $\boldsymbol{f}_{k,l}(\mathbf{q}, \mathbf{v})$ rearranged in a matrix form so that they share a single scaling factor α_k within the product $\mathbf{F}\boldsymbol{\alpha}$. The key observation is that under this lumped parameterization, the SOCP (3) becomes:

$$\begin{aligned} &\min_{\boldsymbol{\alpha}} \frac{1}{2}\mathbf{f}(\boldsymbol{\alpha})^\top [(\mathbf{G} + \mathbf{R})\Delta t]\mathbf{f}(\boldsymbol{\alpha}) + (\bar{\boldsymbol{v}}_f + \boldsymbol{\gamma})^\top \mathbf{f}(\boldsymbol{\alpha}) \\ &\text{s.t. } \boldsymbol{\alpha} \succeq \mathbf{0} \end{aligned} \tag{4}$$

which is a box-constrained QP (BQP) since $\mathbf{f}(\boldsymbol{\alpha}) = \mathbf{F}\boldsymbol{\alpha}$ is affine. In particular, as spring-damper forces lie within the friction cone by closed form construction, the cone constraints $\mathbf{f} \in \mathbb{K}_\mu$ simply become a box constraint $\boldsymbol{\alpha} \succeq \mathbf{0}$.

In the software implementation, the BQP (4) is solved via projected coordinate descent, with each parallel simulation instance running the algorithm in a single thread (i.e. without any intra-simulation parallelism or any cross-thread synchronization). There are two main reasons for this. First, it allows parallelization of QPs with different numbers of decision variables (as each simulation in the batch can have a different number of collision pairs active that survive the OBB prepass). And second, it can handle the case where $\boldsymbol{Q}$ is only positive semidefinite, since this causes a zero diagonal entry $Q_{ii} = 0$, which in turn means the coordinate descent update for the single coordinate i actually sees a linear objective and is hence reduced to a box projection depending on $\text{sign}(c_i)$.

2) Stagger Iterations: After computing the scaling factors $\boldsymbol{\alpha}$, the generalized acceleration under their influence is simply $\mathbf{a} = \mathbf{a}_f + \mathbf{M}^{-1}\bar{\mathbf{J}}^\top \mathbf{F}\boldsymbol{\alpha}$, meaning their contribution to the generalized acceleration can be isolated as $\Delta\mathbf{a}(\boldsymbol{\alpha}) = \mathbf{M}^{-1}\bar{\mathbf{J}}^\top \mathbf{F}\boldsymbol{\alpha}$. With this observation in place, we can extend the spring-damper rescaling formulation from the previous section into the following stagger scheme:

(i) Initialize $\mathbf{f}_{\text{tot}} = \mathbf{0}$, $\mathbf{v}^+ = \mathbf{v}_f$, $\mathbf{F} = \mathbf{F}(\mathbf{q}, \mathbf{v}^+)$, and define the parameterization $\mathbf{f}(\boldsymbol{\alpha}) = \mathbf{F}\boldsymbol{\alpha} + \mathbf{f}_{\text{tot}}$.
(ii) Solve BQP (4) with $\mathbf{f}(\boldsymbol{\alpha}) = \mathbf{F}\boldsymbol{\alpha} + \mathbf{f}_{\text{tot}}$ to obtain $\boldsymbol{\alpha}^*$.
(iii) Update $\mathbf{f}_{\text{tot}} = \mathbf{f}_{\text{tot}} + \mathbf{F}\boldsymbol{\alpha}^*$ and $\mathbf{v}^+ = \mathbf{v}^+ + \Delta\mathbf{a}(\boldsymbol{\alpha}^*)\Delta t$.
(iv) Obtain a new spring-damper force distribution $\mathbf{F} = \mathbf{F}(\mathbf{q}, \mathbf{v}^+)$ under the new velocity $\mathbf{v}^+$.
(v) Iterate (ii)-(iv) until convergence.

In the software implementation, stagger iterations are run using the norm of $\Delta\mathbf{a}(\boldsymbol{\alpha}^*)$ as a convergence criterion.

The intuition behind this staggering scheme is as follows. The original BQP formulation (4) assigns only a single scaling factor per collision pair, even though every collision pair under the proposed collision detection routine produces a dense manifold of multiple contact points. Not having a per-contact point decision variable inherently imposes a restriction on how expressive (i.e. multimodal) the contact force distribution between a collision pair can be, primarily determined by the functional form of the spring-damper model $\boldsymbol{f}_{k,l}(\mathbf{q}, \mathbf{v})$. The staggering scheme addresses this limitation by building the total contact force $\mathbf{f}_{\text{tot}}$ additively across multiple iterations, where the spring-damper force distribution is allowed to readjust at each iteration. And this re-adjustment is based on the contact velocities $\mathbf{v}^+$ that would be observed if one were to do a semi-implicit Euler update with the contact force $\mathbf{f}_{\text{tot}}$ accumulated so far. For example, if the accumulated contact force only pushes out one side of a contact manifold, the closed-form spring-damper expression in the next iteration assigns that side no weight (under the time-stepped velocity $\mathbf{v}^+$ seen by the current stagger iteration, it *is* pushed out). The distribution then concentrates on the remaining side, introducing a new mode.

3) Frictional Correction: As previously described, adding a regularization matrix $\mathbf{R} \neq \mathbf{0}$ inadvertently introduces a $\mathbf{f}^\top\mathbf{R}\mathbf{f}$ term into the optimization objective that scales down contact forces. This means frictional contact forces cannot achieve the exact value needed to maintain complete staticity in a rigid-body system (e.g. zero velocity and acceleration), meaning all contacts eventually slip under constant forces such as gravity. This artifact is present in most commonly used simulators in robotics [7, 8, 10]. The proposed contact dynamics model fixes it via the following additions to the stagger iterations:

- A tangential direction $\hat{\boldsymbol{t}}_i$ is constructed for each contact point i. The software implementation provides two choices, normalizing the accumulated: i) tangential contact force $\mathbf{f}_{\text{tot},i,t}$, or ii) tangential contact velocity $\boldsymbol{J}_i\mathbf{v}_t^+ \in \mathbb{R}^3$.
- For each contact point i, the gap between $\mathbf{f}_{\text{tot},i,t}$ and its Coulomb upper bound $\mu_i\mathbf{f}_{\text{tot},i,n}$ (using the current accumulated normal force) is computed as $\Delta f_{i,t} = \mu_i\mathbf{f}_{\text{tot},i,n} - \mathbf{f}_{\text{tot},i,t}$.
- The frictional correction vectors $\Delta f_{i,t}\hat{\boldsymbol{t}}_i$ are constructed for each contact point, and are re-arranged in a matrix form $\bar{\mathbf{F}}$ so that they share a single scaling factor α_k (per collision pair k) within the product $\bar{\mathbf{F}}\boldsymbol{\alpha}$ (same as spring-damper forces from the previous section).

With this construction in place, let us define the affine parameterization $\bar{\mathbf{f}}(\boldsymbol{\alpha}) = \mathbf{f}_{\text{tot}} + \bar{\mathbf{F}}\boldsymbol{\alpha}$ to use for frictional correction. Note that setting all scaling factors to one results in a contact force $\bar{\mathbf{f}}(\mathbf{1})$ where all points saturate their Coulomb upper-bound (exactly if $\hat{\boldsymbol{t}}_i$ comes from $\mathbf{f}_{\text{tot},i,t}$, only approximately if it comes from $\boldsymbol{J}_i\mathbf{v}^+$). One can then compute the frictional correction that minimizes the system kinetic energy after a semi-implicit Euler update by solving the following BQP:

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \; & \frac{1}{2}\bar{\mathbf{f}}(\boldsymbol{\alpha})^\top(\mathbf{G}\Delta t)\bar{\mathbf{f}}(\boldsymbol{\alpha}) + \bar{\boldsymbol{v}}_f^\top\,\bar{\mathbf{f}}(\boldsymbol{\alpha}) \\ \text{s.t.} \; & \mathbf{0} \preceq \boldsymbol{\alpha} \preceq \mathbf{1} \end{aligned} \tag{5}$$

Overall, this yields the following stagger scheme:

(i) Initialize $\mathbf{f}_{\text{tot}} = \mathbf{0}$, $\mathbf{v}^+ = \mathbf{v}_f$, $\mathbf{F} = \mathbf{F}(\mathbf{q}, \mathbf{v}^+)$, $\bar{\mathbf{F}} = \mathbf{0}$.
(ii) Solve BQP (4) with $\mathbf{f}(\boldsymbol{\alpha}) = \mathbf{F}\boldsymbol{\alpha} + \mathbf{f}_{\text{tot}}$ to obtain $\boldsymbol{\alpha}^*$.
(iii) Update $\mathbf{f}_{\text{tot}} = \mathbf{f}_{\text{tot}} + \mathbf{F}\boldsymbol{\alpha}^*$ and $\mathbf{v}^+ = \mathbf{v}^+ + \Delta\mathbf{a}(\boldsymbol{\alpha}^*)\Delta t$.
(iv) Construct $\bar{\mathbf{F}}$ for frictional correction, using $\mathbf{f}_{\text{tot}}$ or $\mathbf{v}^+$.
(v) Solve BQP (5) with $\mathbf{f}(\boldsymbol{\alpha}) = \bar{\mathbf{F}}\boldsymbol{\alpha} + \mathbf{f}_{\text{tot}}$ to obtain $\boldsymbol{\alpha}^*$.
(vi) Update $\mathbf{f}_{\text{tot}} = \mathbf{f}_{\text{tot}} + \bar{\mathbf{F}}\boldsymbol{\alpha}^*$ and $\mathbf{v}^+ = \mathbf{v}^+ + \Delta\mathbf{a}(\boldsymbol{\alpha}^*)\Delta t$.
(vii) Obtain a new spring-damper force distribution $\mathbf{F} = \mathbf{F}(\mathbf{q}, \mathbf{v}^+)$ under the new velocity $\mathbf{v}^+$.
(viii) Iterate (ii)-(vii) until convergence.

An important observation is that the updates in steps (iii) and (vi) allow the spring-damper re-distribution and re-scaling mechanisms and the frictional correction mechanism to converge in a way that is aware of one another.

4) Validity of the Lumped Parameterization: Whenever $\mathbf{R} \neq 0$, the SOCP (3) is exactly equivalent to the following unconstrained non-linear least-squares problem on the generalized acceleration $\mathbf{a}$ (often called its "primal form" [7, 14]):

$$\begin{aligned} \min_{\mathbf{a}} \; & \frac{1}{2}(\mathbf{a} - \mathbf{a}_f)^\top\mathbf{M}(\mathbf{a} - \mathbf{a}_f) \\ & + \frac{1}{2}\left[\mathbf{P}_{\mathbb{K}_\mu}(\bar{\boldsymbol{a}}_{\text{ref}} - \bar{\mathbf{J}}\mathbf{a})\right]^\top\mathbf{R}^{-1}\left[\mathbf{P}_{\mathbb{K}_\mu}(\bar{\boldsymbol{a}}_{\text{ref}} - \bar{\mathbf{J}}\mathbf{a})\right] \end{aligned} \tag{6}$$

where $\mathbf{P}_{\mathbb{K}_\mu}$ is the closed form projection operator onto the 3D circular cone, and $\bar{\boldsymbol{a}}_{\text{ref}} = \bar{\mathbf{J}}\mathbf{a}_f - \mathbf{r}/\Delta t \in \mathbb{R}^3$. This means even if the apparent dimensionality of the SOCP (3) is $3C$, the intrinsic mechanism simply operates by adjusting the n_v degrees of freedom between all colliding bodies such that a least-squares fit is achieved with respect to contact

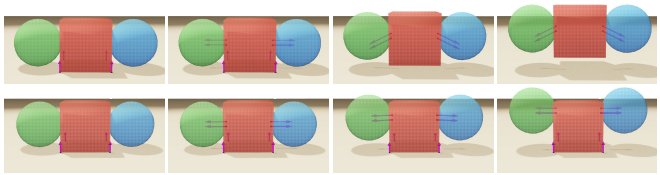


Fig. 4: Antipodal pinching and lifting of a cube, with (top) and without (bottom) the proposed friction correction scheme.

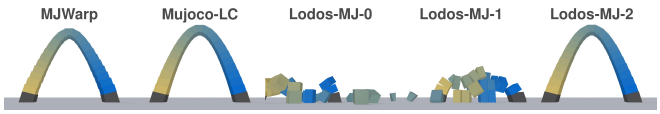


Fig. 5: Final frames from 15-second simulations of a masonry arch.

acceleration references $\bar{\boldsymbol{a}}_{\text{ref}}$ obtained from closed form spring-damper expressions. This means the SOCP formulation (3) also does not make decoupled adjustments between contact points that lie on the same body and are in close proximity (since they share the same underlying degrees of freedom within $\mathbf{a}$). Therefore, the main difference between the well-established SOCP formulation and the lumped parametrization presented here is how they choose to introduce a coupling between proximate contact points: the former couples contact points that share the same degrees of freedom in the multi-body system, and the latter couples contact points that share the same collision pair.

V. Experiments

Let us now present a set of experiments that characterize the proposed simulator across three axes: i) physical faithfulness, ii) throughput, and iii) usefulness for contact-rich control. Throughout all experiments, Lodos-MJ-N denotes the pipeline proposed in this paper, optimizing the MuJoCo convention for $\mathbf{R}$ and γ, with N stagger iterations ($N = 0$ reduces to combining the unscaled spring-damper forces obtained by the method of [38, 37] with our collision detection). MuJoCo-CPU and MJWarp denote the default CPU implementation and the GPU implementation [2] in NVIDIA Warp [39]; MuJoCo-LC combines our collision detection with MuJoCo contact dynamics. For all experiments and simulators, we use a matched 2 ms timestep size which is the MuJoCo default value. Simulator settings, including Lodos memory capacities and MJWarp contact and constraint limits, are tuned for maximum performance in each experiment. All experiments run on a single NVIDIA RTX 4080 GPU. Supplementary videos accompany all experiments.

A. Physical Faithfulness

1) Effectiveness of the Friction Correction Scheme: Fig. 4 conducts a didactic experiment where two spherical fingers with $r = 0.5m$ are pinching a $1m$ $10kg$ cube, lifting it up 0.3 m over a 1.2 s interval with a speed of 0.25 m/s, and holding it for $5s$. The spheres use motor actuators along the horizontal axis applying a $100N$ force each, and position controllers with $k_p = 1e^4$ and $k_v = 200$ in the vertical axis. The friction coefficient is $\mu = 0.6$, meaning the Coulomb upper-bound for the tangential force is $120N$ compared to the $100N$ weight of the cube. The experiment compares

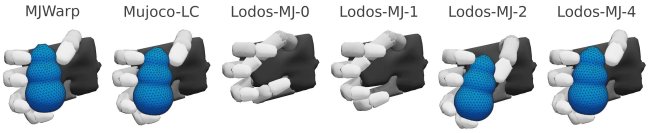


Fig. 6: Final frames from a 5 s simulation in which a position controller, with parameters optimized in MuJoCo to maintain a delicate prismatic grasp, is transferred directly to Lodos.

the closed form spring-damper model of [38, 37] with and without our frictional correction scheme. Without frictional correction, the deliberately small rising velocity of the fingers prevents the spring-damper model from generating sufficient frictional force to lift the cube (the maximum observed value is 8.22 N). With our friction correction scheme, the fingers exert an exact $120N$ frictional force on the cube during the first $114ms$ of the pickup resulting in 5.97 mm of slip in between, after which point the contact sticks. During the $5s$ hold afterwards, a mean creep velocity of $-0.0014mm/s$ is observed under $g = 9.81\,m/s^2$ gravitational acceleration (likely due to the finite tolerance and 32-bit precision of our BQP solver). These results show that the proposed frictional correction scheme operates as intended.

2) Stability of a Masonry Arch: Fig. 5 shows a masonry arch simulation with 25 stones. As can be seen, unscaled spring-damper forces alone cannot achieve sufficient physical fidelity to hold the arch upright. Doing one stagger iteration to rescale the spring-damper forces diminishes the severity, and by two stagger iterations the arch is stable. Stability of the MuJoCo-LC baseline further confirms the validity of the proposed collision detection method. In particular, MJWarp's reliance on GJK+EPA contact points that jump discontinuously across timesteps (due to parallel-face collisions) results in visible slipping between the stones, an artifact fixed by the MuJoCo-LC and Lodos-MJ-2 configurations.

3) Locomotion with a Bipedal Robot: Fig. 1 shows successful sim-to-sim transfer for the "Microduck" platform, where policies trained in MJWarp (copied directly from the open-source Microduck repository [40]) are deployed zero-shot in Lodos to execute diverse locomotion behaviors. This further demonstrates the physical faithfulness of Lodos.[1]

4) Prismatic Grasp with an Anthropomorphic Hand: Fig.6 shows another successful sim-to-sim transfer, this time of a delicate four-finger prismatic grasp. The grasping pose and the position control targets that squeeze it are optimized in MuJoCo using the Dexonomy framework [41], and are then directly transferred to Lodos. The results show the same pattern as the arch experiment, where physical faithfulness increases with further stagger iterations.

B. Throughput Characterization

1) Simulating Complex Collision Geometry: Figure 7 and Table I evaluate batches of 20 armadillo meshes falling onto a plane, with randomized initial configurations per instance. Lodos-MJ-1 represents each armadillo with 451 vertices and 100 spheres; MJWarp uses 34 convex polytopes. Throughput is averaged over the 1–4 s interval of a 10 s simulation, when impacts and contacts dominate. This excludes contact-free and

[1]Please refer to the supplementary for videos of all simulations.

TABLE I: Simulation throughput across batch sizes, and speed-up over MJWarp.

Scene	Simulator	N_s	Step time per world (µs) at batch size B									Speed-up over MJWarp	
			2^0	2^3	2^6	2^9	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}	Whole step	Contact pipeline
In-hand reorientation	MJWarp		904	147	21.8	4.08	**2.75**	—	—	—	—		
	LODOS-MJ-N	0	523	74.3	10.7	2.12	0.438	0.317	**0.299**	0.313	0.346	9.19×	14.0×
		1	656	102	15.9	3.07	0.590	0.409	**0.367**	0.374	0.419	7.49×	11.0×
		2	799	136	21.3	3.91	0.733	0.501	0.440	**0.437**	0.495	6.29×	9.90×
20-armadillo drop	MJWarp		1335	216	40.1	**17.9**	—	—	—	—	—		
	LODOS-MJ-N	0	270	54.4	14.7	3.02	0.942	**0.824**	0.879	—	—	21.7×	24.8×
		1	404	84.1	24.7	4.96	1.33	**1.21**	1.40	—	—	14.8×	16.7×
		2	580	134	39.5	7.76	1.96	**1.73**	2.00	—	—	10.3×	11.1×

Numerical results for the throughput characterization experiments. **Left:** Wall-clock time of one simulation step divided by the batch size. Each entry is the median over three batches. **Right:** Speed-up over MJWarp with each simulator at its peak batch size, for the whole step and for the contact pipeline alone (i.e. the latter omits the shared unconstrained rigid-body dynamics pass). N_s denotes the number of stagger iterations.

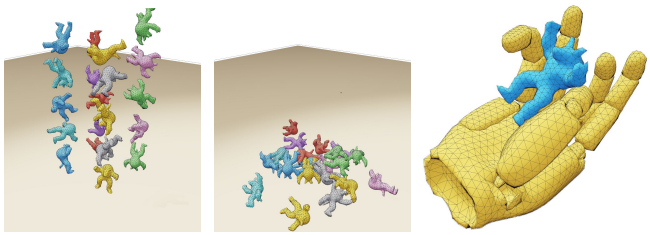


Fig. 7: Throughput characterization experiments that involve complex mesh-mesh collisions. **Left**: 20 armadillos falling on a plane. **Right**: A Sharpa Wave hand holding an armadillo.

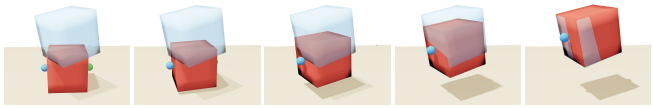


Fig. 8: Trajectory optimization experiment using iLQR, with linearizations of the system dynamics estimated via zeroth-order randomized smoothing using Lodos-MJ-1.

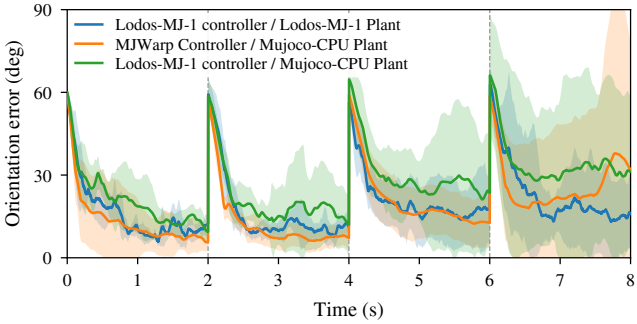


Fig. 9: Stochastic MPC experiment where MPPI is used to manipulate an armadillo in hand with a Sharpa Wave hand, tracking four consecutive goal orientations. The plots show the mean and standard deviation across 5 randomly sampled goal sequences.

settled phases where solver warm starts confound comparison. Lodos maintains physical faithfulness while achieving an order of magnitude higher peak throughput.

2) In-Hand Manipulation with Detailed Meshes: We also benchmark a Sharpa Wave hand manipulating the same armadillo (Fig. 7, Table I). Lodos-MJ-1 represents the hand with 3,479 vertices across 23 bodies, without any inscribed spheres (as placing them only within the armadillo is sufficient). MJWarp in turn uses 87 convex polytopes for the entire hand. To evaluate throughput across varying batch sizes, we run the MPPI controller from S.V-C.2, tasked with tracking four random goals sampled to be 60° apart along a random rotation vector, to generate a full contact-rich manipulation trajectory. We then extract the 2,500 states with the highest number of contacts. Each batch with size B across the ladder is constructed by sampling B states out of these 2500 with replacement. The reported values are averaged over three repetitions. The results demonstrate that the proposed framework again achieves an order of magnitude higher maximum throughput compared to MJWarp.

C. Usefulness for Contact-Rich Control

1) iLQR with Randomized Smoothing: Fig.8 shows a trajectory optimization example using iLQR, where linearizations use the zeroth-order randomized smoothing method of [6]. The analogous experiment in [6] pushes a cube with a sphere on a 2D plane (i.e. each with two DoF) under quasistatic dynamics without gravity or ground friction. Here, two position-controlled three-DoF spheres ($r = 0.1$ m, 1 kg) pinch and lift a six-DoF cube (1 m, 5 kg) by 1 m with a random 60° target rotation, under full second-order dynamics and gravity. The state space (generalized position and velocity) is 24 dimensional and the rotational degree of freedom uses the so(3) parameterization. Controls are 6 dimensional, and the parameterization splits the $2s$ trajectory into 20 knot points interpolated with a zero-order hold. The objective and Q, R matrices follow [6]. Noise level for randomized smoothing is picked via grid search on the iLQR termination cost.

The experiment involves two configurations. In the first one, both the dynamics linearization and the iLQR rollouts (and line-search) use Lodos-MJ-1. The second configuration is mixed, where linearization uses Lodos-MJ-1 and rollouts use MJWarp. Both successfully reach the goal configuration with very similar precision, with the former achieving a $0.75mm$ translational and 0.13° rotational error, and the latter 10.5 mm and 1.24° respectively. This demonstrates that the proposed contact model can be used to estimate gradients that are useful for trajectory optimization, which remain informative even when transferred across different contact models.

2) In-Hand Manipulation with MPPI: Fig. 9 shows the results of an in-hand manipulation experiment with the same Sharpa Wave hand and armadillo from S.V-B.2. The SO(3) pose of the armadillo needs to track four consecutive goal poses that differ by a 60° rotation along a random axis in 3D. Each goal is held fixed for a quarter of the overall

8 s manipulation sequence. All experiments use an MPPI controller with 64 parallel rollouts and without any domain randomization, as implemented by the hydrax framework [5] (where all the remaining MPPI parameters are directly carried over from the analogous in-hand manipulation experiment in hydrax). There are three simulator configurations which are used for the plant and for obtaining the rollouts of the MPPI controller: i) Lodos-MJ-1 for both the controller and the plant, ii) MJWarp for the controller and MuJoCo-CPU for the plant, and iii) Lodos-MJ-1 for the controller and MuJoCo-CPU for the plant. The tracking performance plots and the supplementary videos demonstrate that: i) Lodos-MJ-1 can simulate extremely fast, contact-rich manipulation behavior without artifacts, and ii) actions optimized in Lodos-MJ-1 successfully transfer to MuJoCo-CPU.

VI. Limitations and Future Work

An important clarification about the proposed framework is that it is designed to maximize *throughput* in the presence of *complex mesh-mesh collisions*. For small (e.g. less than ~ 256) or medium-sized (e.g. $\sim 256 - 1024$) batches, the rollout APIs of MuJoCo-CPU and MJWarp remain the best options respectively. Only by allowing the batch size to be increased does Lodos gain an edge in throughput, and mainly thanks to its collision detection and contact solver routines being designed specifically such that they scale well into this regime. Similarly, when it is sufficient to approximate geometries by primitives (e.g. spheres, capsules, boxes), MJWarp still has the edge in terms of throughput. Another limitation is that the software implementation does currently not support automatic differentiation via Nvidia Warp.

VII. Conclusion

We presented a novel simulation approach built from the ground up to facilitate research on contact-rich manipulation and control. We experimentally validated its merits along three axes: i) physical faithfulness, ii) computational throughput, and iii) utility for control. Promising directions for future work include extending the framework to support automatic differentiation and leveraging it to develop novel optimal control algorithms for contact-rich settings.

Acknowledgments

Software development was assisted by Claude Opus 5.

References

[1] V. Makoviychuk, L. Wawrzyniak, Y. Guo *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," in *NeurIPS Datasets and Benchmarks*, 2021.

[2] Google DeepMind and NVIDIA, "MuJoCo Warp (MJWarp)."

[3] A. Serifi, R. Grandia, E. Knoop *et al.*, "Vmp: Versatile motion priors for robustly tracking motion on physical characters," in *Comput. Graph. Forum*, vol. 43, no. 8. Wiley Online Library, 2024.

[4] A. Allshire *et al.*, "Transferring dexterous manipulation from gpu simulation to a remote real-world trifinger," in *IROS*. IEEE, 2022.

[5] V. Kurtz, "Hydrax: Sampling-based model predictive control on gpu with jax and mujoco mjx," 2024.

[6] T. Pang, H. T. Suh, L. Yang, and R. Tedrake, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Trans. Robot.*, 2023.

[7] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *IEEE IROS*, 2012.

[8] R. Tedrake and the Drake Team, "Drake: Model-based design and verification for robotics," 2019.

[9] J. Carpentier, L. Montaut, and Q. Lidec, "From compliant to rigid contact simulation: a unified and efficient approach," in *RSS*, 2024.

[10] V. Tsounis, G. Maloisel, C. Schumacher *et al.*, "Kamino: Gpu-based massively parallel simulation of multi-body systems with challenging topologies," *arXiv:2603.16536*, 2026.

[11] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, pp. 193–203, 1988.

[12] D. Gregorius, "Robust contact creation for physics simulations," in *GDC*, 2015.

[13] S. Andrews, K. Erleben, and Z. Ferguson, "Contact and friction simulation for computer graphics," in *ACM SIGGRAPH Courses*, 2022, pp. 1–172.

[14] A. M. Castro, F. N. Permenter, and X. Han, "An unconstrained convex formulation of compliant contact," *IEEE Trans. Robot.*, 2023.

[15] Q. Le Lidec, W. Jallet, L. Montaut, I. Laptev, C. Schmid, and J. Carpentier, "Contact models in robotics: a comparative analysis," *IEEE Trans. Robot.*, 2023.

[16] M. Anitescu, "Optimization-based simulation of nonsmooth rigid multibody dynamics," *Math. Prog.*, vol. 105, p. 113, 2006.

[17] D. E. Stewart, "Rigid-body dynamics with friction and impact," *SIAM Rev.*, vol. 42, no. 1, pp. 3–39, 2000.

[18] G. Van Den Bergen, *Collision detection in interactive 3D environments*. CRC Press, 2003.

[19] M. S. Corporation, "Adams (automated dynamic analysis of mechanical systems)."

[20] K. H. Hunt and F. R. Crossley, "Coefficient of restitution interpreted as damping in vibroimpact," *J. Appl. Mech.*, vol. 42, no. 2, pp. 440–445, 1975.

[21] V. Kurtz and A. Castro, "Cenic: Convex error-controlled numerical integration for contact," *arXiv:2511.08771*, 2025.

[22] D. E. Stewart and J. C. Trinkle, "An implicit time-stepping scheme for rigid body dynamics with inelastic collisions and coulomb friction," *Int. J. Numer. Methods Eng.*, vol. 39, no. 15, pp. 2673–2691, 1996.

[23] simple-Robotics, https://github.com/simple-Robotics/simple, 2024.

[24] C. Habermann and F. Kindermann, "Multidimensional spline interpolation: Theory and applications," *Comput. Econ.*, vol. 30, no. 2, pp. 153–169, 2007.

[25] O. Beker, A. R. Geist, A. Paulus, N. Gürtler, J. Shi, S. Calinon, and G. Martius, "Smoothly differentiable and efficiently vectorizable contact manifold generation," *arXiv:2602.20304*, 2026.

[26] B. Heijden, "Bubblify: Interactive URDF spherization tool," github.com/bheijden/bubblify, 2024.

[27] S. Coumar, G. Chang, N. Kodkani, and Z. Kingston, "Foam: A tool for spherical approximation of robot geometry," 2025.

[28] O. Beker, A. R. Geist, A. Paulus, and G. Martius, "Novel algorithms for smoothly differentiable and efficiently vectorizable contact manifold construction," *arXiv:2604.17538*, 2026.

[29] G. M. Morton, "A computer oriented geodetic data base and a new technique in file sequencing," IBM Ltd., Ottawa, Canada, Tech. Rep., 1966.

[30] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer v3: Simpler, faster, stronger," in *CVPR*, 2024.

[31] J. Ragan-Kelley, "Lecture 7: Data parallel primitives (part 1)," Lecture slides, MIT 6.S894: Accelerated Computing, October 2025.

[32] NVIDIA, "Cuda programming guide, release 13.2," 2026.

[33] K. L. Johnson, *Contact mechanics*. Cambridge Uni. press, 1987.

[34] P. Kidger, "On Neural Differential Equations," Ph.D. dissertation, University of Oxford, 2021.

[35] E. Todorov, "Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in mujoco," in *ICRA*. IEEE, 2014, pp. 6054–6061.

[36] V. Kurtz, A. Castro, A. O. Önol, and H. Lin, "Inverse dynamics trajectory optimization for contact-implicit model predictive control," *Int. J. Robot. Res.*, 2025.

[37] C. Borse, Z. Xie, W. Huang, and W. Jin, "Comfree-sim: A gpu-parallelized analytical contact physics engine for scalable contact-rich robotics simulation and control," *arXiv:2603.12185*, 2026.

[38] W. Jin, "Complementarity-free multi-contact modeling and optimization for dexterous manipulation," in *RSS*, 2025.

[39] M. Macklin, "Warp: A High-performance Python Framework for GPU Simulation and Graphics," 2022, NVIDIA GTC.

[40] Pollen Robotics, "microduck_rl: Reinforcement learning environments and control policies for the microduck biped robot," https://github.com/pollen-robotics/microduck_rl, 2024.

[41] J. Chen, Y. Ke, L. Peng, and H. Wang, "Dexonomy: Synthesizing all dexterous grasp types in a grasp taxonomy," in *RSS*, 2025.